

Vietnam Administrative Address Converter

Technical Reference Documentation

v1.0.1 · github.com/trng28/address-converter

98.5%

Conversion rate

74.2%

Ward remapped

1.5%

Unresolved

99.5%

District removed

1. Overview

Vietnam's 2025 administrative reform eliminated the district level across all 63 provinces, merging and renaming thousands of wards. The address-converter library provides programmatic parsing and conversion of Vietnamese address strings between the legacy 3-level structure (province / district / ward) and the new 2-level structure (province / ward).

1.1 Design goals

- Zero external runtime dependencies - standard library only.
- Parse free-form text without requiring a structured input format.
- Return deterministic, structured JSON with confidence scores and warnings.
- Support both single-address and bulk CSV workflows.
- Maintain a configurable LRU cache for high-throughput batch processing.

1.2 Architecture

The library follows a layered pipeline:

Processing pipeline

Input text → **normalize.py** → stripped, lowercased, diacritics removed

Normalized text → **text.py** → admin prefix detected, street extracted

Parsed components → **data.py** → looked up in admin data indexes

Matched codes → **conversion.py** → old ward mapped to new ward

Conversion result → **converter.py** → assembled into output JSON

2. Module reference

All public symbols are importable from the top-level package via `from src import ...`

2.1 `src/__init__.py` - Public API

The package entry point exposes the primary conversion function.

`auto_convert_address`

```
from src import auto_convert_address

result = auto_convert_address(
    "Phuong An Hai Nam, Quan Son Tra, Da Nang"
)
```

Accepts a free-form Vietnamese address string and returns a fully resolved conversion result. Internally calls

Parameters

Parameter	Type	Default	Description
<code>address</code>	<code>str</code>	—	Free-form address string. Unicode and ASCII both accepted.

Returns `dict`

See Section 4 (Output Schema) for the complete field reference.

2.2 `src/converter.py` - Converter

Core orchestration class. Holds references to the loaded data indexes and calls the parse and conversion pipeline.

`create_converter()`

```
from src.converter import create_converter

converter = create_converter() # loads data once
result = converter.parse_address_text("...")
result = converter.parse_new_address_text("...")
```

Factory function. Initialises data indexes (ward, district, province, mapping tables) and returns a

`Converter.parse_address_text(text)`

Parameter	Type	Default	Description
<code>text</code>	<code>str</code>	—	Any address string. The method auto-detects old vs. new structure.

Behaviour

- Normalizes input via
- Calls

- Runs conversion pipeline and returns result dict.

Converter.parse_new_address_text(text)

Same signature as `parse_address_text`. Accepts an address already in the new (post-2025) format and validates or resolves it against the new ward index without running the old-to-new mapping step.

2.3 src/text.py - Address parser

Responsible for detecting the boundary between the street component and the administrative suffix, then splitting the raw address into its constituent parts.

`_find_admin_start_index(text)`

```
from src.text import _find_admin_start_index

idx = _find_admin_start_index(
    "Đường tỉnh 950, Xã Khánh Bình, Huyện An Phú, Tỉnh An Giang"
)
# Returns 16 (position of "Xã")
```

Returns the character index at which the first administrative prefix begins. Recognises both Unicode and ASCII-normalized variants (e.g.

Prefix priority order (highest to lowest)

- Thị trấn / thị trấn
- Thị xã / thị xã
- Thành phố / thành phố
- Phường / phường, Quận / quận, Huyện / huyện
- Tỉnh (only before a word with diacritics, not bare numbers)
- Xã (only before a word with diacritics)
- tp (word-boundary match)

Edge case: "Đường tỉnh 950"

"tỉnh" in "Đường tỉnh 950" is part of the street name, not an admin prefix. The regex requires tỉnh to be followed by a proper noun (word containing Vietnamese diacritics), so "tỉnh 950" is correctly skipped. The match lands on "Xã" at index 16.

2.4 src/normalize.py - Text normalization

Converts address strings to a canonical form used internally for lookups and matching.

`normalize_address_text(text)`

```
from src.normalize import normalize_address_text

normalize_address_text("Phường Thảo Điện")
# → "phuong thao dien"
```

Parameter	Type	Default	Description
<code>text</code>	<code>str</code>	<code>—</code>	Raw address component or full address string.

Applies in sequence: lowercase → NFKD decomposition → strip combining diacritics → strip extra whitespace. The output is suitable for fuzzy matching and index lookups.

2.5 `src/data.py` - Data layer

Loads and exposes the administrative data at module import time. Provides four indexes built from the embedded JSON data files:

Field	Type	Description
<code>DEFAULT_DATA.mapping.rows</code>	<code>list[dict]</code>	Old-to-new ward mapping rows. Each row contains old province/district/ward codes and the corresponding new ward code.
<code>DEFAULT_DATA.newWards</code>	<code>dict</code>	All new ward objects keyed by new ward code.
<code>DEFAULT_DATA.newProvinces</code>	<code>dict</code>	All new province objects keyed by province code.
<code>DEFAULT_DATA.oldProvinces</code>	<code>dict</code>	Legacy province objects keyed by old province code.

2.6 `src/conversion.py` - Conversion engine

Implements the strategy for matching a parsed old address to a new ward. Uses a tiered match strategy, trying progressively looser constraints until a confident match is found or the result is marked unmatched.

Match strategy hierarchy

- Province_district_ward_name - exact match on all three codes/names (confidence 0.98)
- province_district - ward name not matched; returns top candidate with warning (confidence 0.75)
- province_only - district also unmatched; low confidence match (confidence 0.50)
- unmatched - no mapping found; returns null conversion result

2.7 `src/constants.py` - Configuration

Field	Type	Description
<code>PARSER_VERSION</code>	<code>str</code>	Semantic version string embedded in every response meta block.
<code>MAPPING_VERSION</code>	<code>str</code>	Version tag of the administrative mapping data (e.g. "10_25").
<code>DEFAULT_CACHE_SIZE</code>	<code>int</code>	Default LRU cache size for batch CSV processing (4096).

2.8 src/utils.py - Shared utilities

Internal helpers used across modules. Not part of the public API.

- Slug generation (diacritics removed, spaces replaced with hyphens).
- LRU cache decorator used by the converter for batch workloads.
- Confidence score arithmetic.

3. Command-line interface

3.1 Single address

```
python main.py --address "Đường Trần Nãi, Phường Thảo Điện, TP Thủ Đức, Thành phố Hồ Chí Minh"
```

Parameter	Type	Default	Description
--address	str	—	Free-form address string to convert.
--pretty	flag	true	Pretty-print JSON output (default on).
--no-pretty	flag	false	Compact single-line JSON output.

3.2 Batch CSV

```
python main.py \  
--input-csv data/vietnam_full_address.csv \  
--output-csv data/vietnam_full_address.converted.csv \  
--address-column full_address \  
--cache-size 4096
```

Parameter	Type	Default	Description
--input-csv	str	—	Path to input CSV file.
--output-csv	str	—	Path to write converted CSV file.
--address-column	str	—	Column name containing address strings.
--cache-size	int	4096	LRU cache size. Larger values improve throughput on repeated addresses.

4. Output schema

Every conversion returns a JSON object with the following top-level fields:

Field	Type	Description
<code>input_type</code>	<code>str</code>	"old" or "new" — detected address structure.
<code>normalized_text</code>	<code>str</code>	Reconstructed address string after conversion.
<code>parsed</code>	<code>dict</code> <code>null</code>	Resolved province / district / ward from the input. Null if parsing failed.
<code>converted_new</code>	<code>dict</code> <code>null</code>	New province and ward objects. Null if conversion failed.
<code>conversion</code>	<code>dict</code>	Full conversion record including strategy, confidence, warnings, and timing.
<code>old_candidates</code>	<code>list</code>	Alternative old-address matches considered before the selected result.
<code>old_first</code>	<code>dict</code> <code>null</code>	The highest-ranked old-address candidate when multiple are found.

4.1 parsed

Field	Type	Description
<code>province_name</code>	<code>str</code>	Full name with type, e.g. "Thành Phố Hồ Chí Minh".
<code>district_name</code>	<code>str</code>	Full name with type, e.g. "Thành Phố Thủ Đức".
<code>ward_name</code>	<code>str</code>	Full name with type, e.g. "Phường Thảo Điền".
<code>province_code</code>	<code>str</code>	Legacy province code.
<code>district_code</code>	<code>str</code>	Legacy district code.
<code>ward_code</code>	<code>str</code>	Legacy ward code.

4.2 converted_new

Field	Type	Description
<code>province.code</code>	<code>str</code>	New province code (same as legacy in most cases).
<code>province.name_with_type</code>	<code>str</code>	Canonical province name, e.g. "Thành phố Hồ Chí Minh".
<code>ward.code</code>	<code>str</code>	New ward code.
<code>ward.name_with_type</code>	<code>str</code>	New ward name with type prefix.

<code>ward.path_with_type</code>	str	Full address path: "Phường An Khánh, Thành phố Hồ Chí Minh".
<code>ward.slug</code>	str	URL-safe slug, e.g. "an-khanh".

4.3 conversion

Field	Type	Description
<code>status</code>	str	"matched" "partial" "unmatched".
<code>confidence</code>	float	Match confidence (0.0–1.0). 0.98 = full match; 0.75 = district-level; 0.50 = province-level.
<code>match_level</code>	str	Granularity: "province_district_ward_name", "province_district", or "province_only".
<code>match_strategy</code>	str	Internal strategy key used to find the result.
<code>warnings</code>	list	List of warning strings. Empty on a clean match.
<code>meta.elapsed_ms</code>	float	Wall-clock time for the conversion, in milliseconds.
<code>meta.parser_version</code>	str	Version of the parser (from constants.py).
<code>meta.mapping_version</code>	str	Version of the mapping data (from constants.py).

5. Error handling and edge cases

5.1 Unresolved addresses

When no mapping is found, `converted_new` is `null` and `conversion.status` is `"unmatched"`. The `parsed` field may still contain the legacy province/district/ward if parsing succeeded.

5.2 Low-confidence results

Results with `confidence < 0.75` indicate the ward could not be uniquely identified. Callers should inspect `conversion.warnings` and `old_candidates` before accepting the result.

5.3 Known ambiguous cases

- "Đường tỉnh N" — street names containing the word "tỉnh" followed by a number are correctly excluded from admin prefix detection.
- Khmer-script characters in street names are passed through without modification.
- Duplicate prefixes (e.g. "Phường Phường 1") should be normalized before passing to the converter.
- Addresses from Thừa Thiên Huế and Bình Định have elevated unresolved rates due to mapping table coverage gaps.

6. Performance

Benchmark — 79,931 addresses (vietnam-address-collection dataset)
Conversion rate: 98.5% (78,738 / 79,931)
Ward remapped: 74.2% (59,270 rows)
Ward unchanged: 24.4% (19,468 rows)
District removed: 99.5% (78,344 / 78,738 converted)
Unresolved: 1.5% (1,193 rows - Thừa Thiên Huế 568, Bình Định 454, others 171)

6.1 Cache tuning

The `--cache-size` flag controls the LRU cache on the address lookup step. For datasets with many repeated wards (e.g. city-level address lists), larger cache sizes provide near-linear speedup. The default of 4096 entries covers most province-level datasets.

6.2 Throughput

On a single core (Python 3.10, M-series equivalent), expect approximately:

- ~20 µs / address with a warm cache (repeated ward inputs).
- ~100 µs / address for unique addresses requiring full lookup.
- ~80,000 addresses / second throughput at cache-size 4096 on a diverse dataset.

7. Data sources

Administrative data is embedded in the package and reflects the mapping table version

Field	Type	Description
Old wards	JSON	~10,000 ward records with province/district codes from the pre-2025 structure.
New wards	JSON	~3,300 ward records reflecting the post-2025 two-level structure.
Mapping table	JSON	Row-indexed mapping from each old (province, district, ward) triple to the new ward code.
Provinces	JSON	63 province/city records for both old and new structures.

Validation dataset: [trucmtnguyen/vietnam-address-collection](#) on Hugging Face - 79,931 real-world Vietnamese addresses across all 63 provinces.

8. Changelog

Version	Date	Changes
1.0.1	2026-06-27	Initial release. Mapping version 10_25. 98.5% conversion rate on 79,931-record benchmark.

9. License

Apache 2.0 - see LICENSE file in the repository root.

Dataset: [trucmtnguyen/vietnam-address-collection](#) - see Hugging Face repository for dataset license.